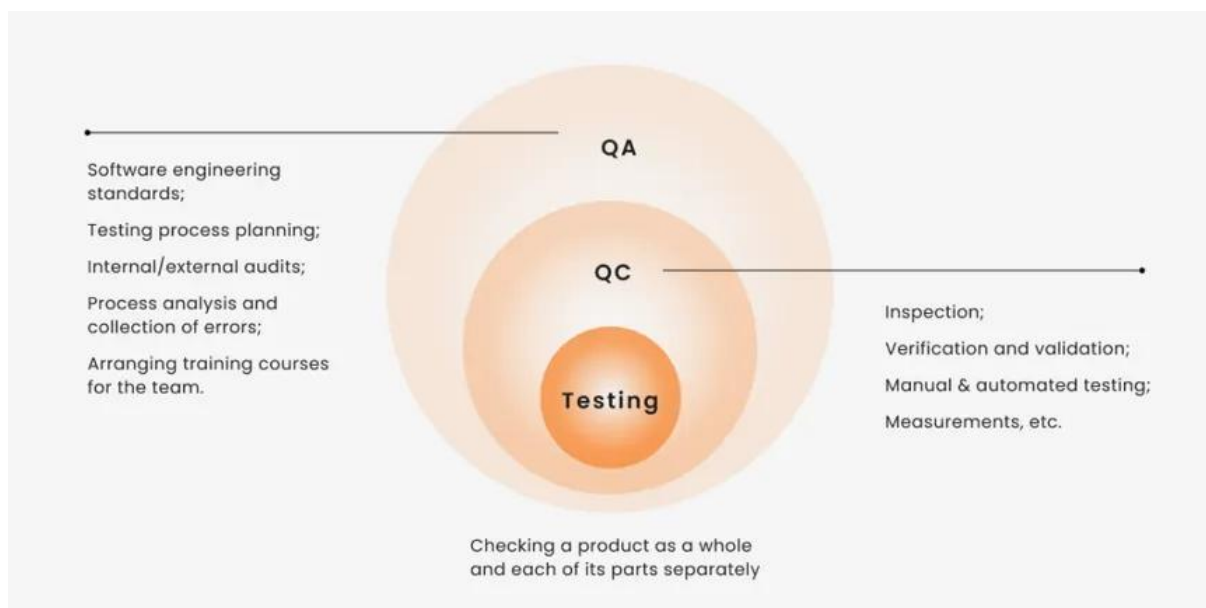


CS341: Software Testing & Quality Assurance

Assignment One Writeup

Semester 2, 2026



Group: 5

Group Name: The Dynamic Duo

The Project Team

Student Name	Student ID
Aryan Singh	S11230987
Shivan Prasad	S11231502

Table of Contents

Section A: Unit Testing with Junit & JaCoCo.....	3
A. section-a-employee-rating	3
B. section-b-overstay-charge	4
JaCoCo Testing Logs	5
Section B – Test Design, Execution and Defect Logging.....	7
Task 4 – Test case identification and Unit Test Plan	7
4.1 Test Design Approach	7
4.2 Equivalence Partitioning	7
4.3 Boundary Value Analysis	8
4.4 Unit Test Plan	9
Conclusion	9

Section A: Unit Testing with JUnit & JaCoCo

A. section-a-employee-rating

These screenshots show the JUnit testing results for the `employee rating` test cases, and the command “`mvn clean test`” was used to get these results, and as shown all our test cases passed with zero failures.



The screenshots show the execution of the command `mvn clean test` in an IDE. The first screenshot shows the command being executed in the terminal. The second screenshot shows the JUnit test results, indicating that all tests passed. The third screenshot shows the JaCoCo test results, indicating that all tests passed and the code coverage is 100%.

```
Active code page: 65001

C:\Users\aryan\OneDrive\Desktop\S2 2026\ASG\CS341\A1\CS341_A1_Java_Starter>cd "c:\Users\aryan\OneDrive\Deskt
p\S2 2026\ASG\CS341\A1\CS341_A1_Java_Starter\CS341_A1_Starter\section-a-employee-rating"

c:\Users\aryan\OneDrive\Desktop\S2 2026\ASG\CS341\A1\CS341_A1_Java_Starter\CS341_A1_Starter\section-a-employe
e-rating>mvn clean test
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
[INFO] Scanning for projects...
[INFO]
[INFO] -----< fj.usp.cs341:employee-rating >-----
[INFO] Building CS341 Section A - Employee Performance Rating System 1.0.0
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ employee-rating ---
[INFO] Skipping JaCoCo execution because property jacoco.skip is set.
[INFO] argLine set to empty
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @
employee-rating ---
[INFO] skip non existing resourceDirectory c:\Users\aryan\OneDrive\Desktop\S2 2026\ASG\CS341\A1\CS341_A1_Java
_Starter\CS341_A1_Starter\section-a-employee-rating\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ employee-rating ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug release 17] to target\classes
[INFO]

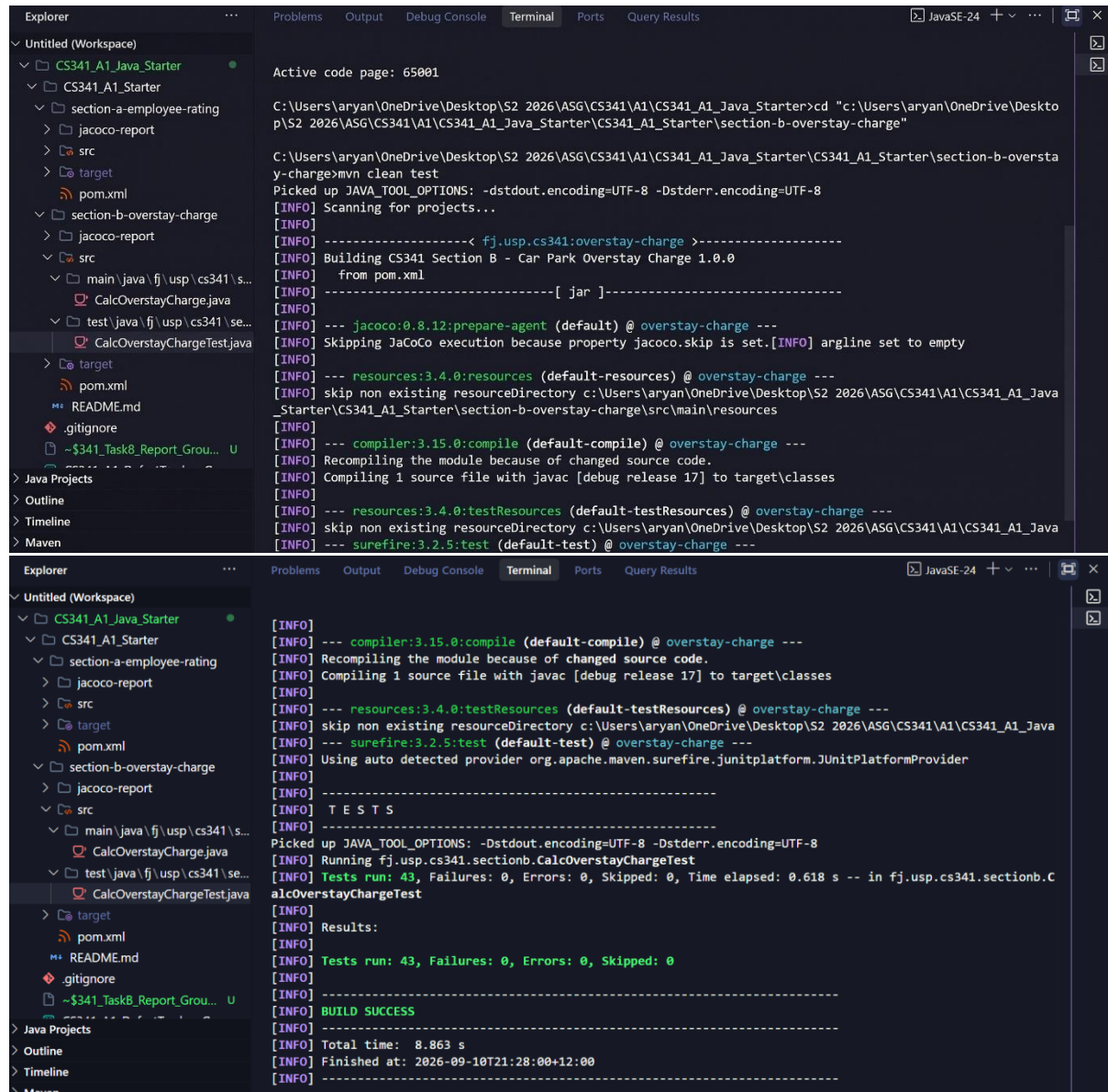
[INFO] --- compiler:3.15.0:compile (default-compile) @ employee-rating ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug release 17] to target\classes
[INFO]

[INFO] --- resources:3.4.0:testResources (default-testResources) @ employee-rating ---
[INFO] skip non existing resourceDirectory c:\Users\aryan\OneDrive\Desktop\S2 2026\ASG\CS341\A1\CS341_A1_Java
_Starter\CS341_A1_Starter\section-a-employee-rating\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ employee-rating ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug release 17] to target\test-classes
[INFO]

[INFO] --- surefire:3.2.5:test (default-test) @ employee-rating ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$SummaryTests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.149 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$SummaryTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$NegativeTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$NegativeTests$InvalidScoreTests
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.269 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$NegativeTests$InvalidScoreTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$NegativeTests$InvalidNameTests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$NegativeTests$InvalidNameTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$NegativeTests$InvalidNameTests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$NegativeTests$InvalidNameTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$BoundaryValueAnalysisTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$BoundaryValueAnalysisTests$ThresholdBoundaries
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.057 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$BoundaryValueAnalysisTests$ThresholdBoundaries
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$EquivalencePartitioningTests
[INFO] Running fj.usp.cs341.sectiona.EmployeeTest$ConstructorAndGetterTests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in fj.usp.cs341.sectiona.Em
ployeeTest$ConstructorAndGetterTests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.773 s -- in fj.usp.cs341.sectiona.Em
ployeeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 42, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 8.671 s
[INFO] Finished at: 2026-09-10T21:26:34+12:00
[INFO]
[INFO] -----
```

B. section-b-overstay-charge

These following screenshots are for the testing for the [section-b-overstay-charge](#) test cases, and we used the command “mvn clean test” to get these results. And as shown all our test cases passed with zero failures.



The image displays two screenshots of an IDE terminal window, showing the execution of Maven commands and the resulting output for the 'section-b-overstay-charge' module.

Top Screenshot: The terminal shows the execution of the command `cd "c:\Users\aryan\OneDrive\Desktop\S2 2026\ASG\CS341\A1\CS341_A1_Java_Starter\section-b-overstay-charge"` followed by `mvn clean test`. The output indicates that the project is being scanned for projects, and the build process is initiated. The output shows the following steps:

- Building CS341 Section B - Car Park Overstay Charge 1.0.0 from pom.xml
- Compiling 1 source file with javac [debug release 17] to target\classes
- Running tests using JUnit 4.12.0 and JUnit Platform 1.8.2
- Tests run: 43, Failures: 0, Errors: 0, Skipped: 0
- BUILD SUCCESS

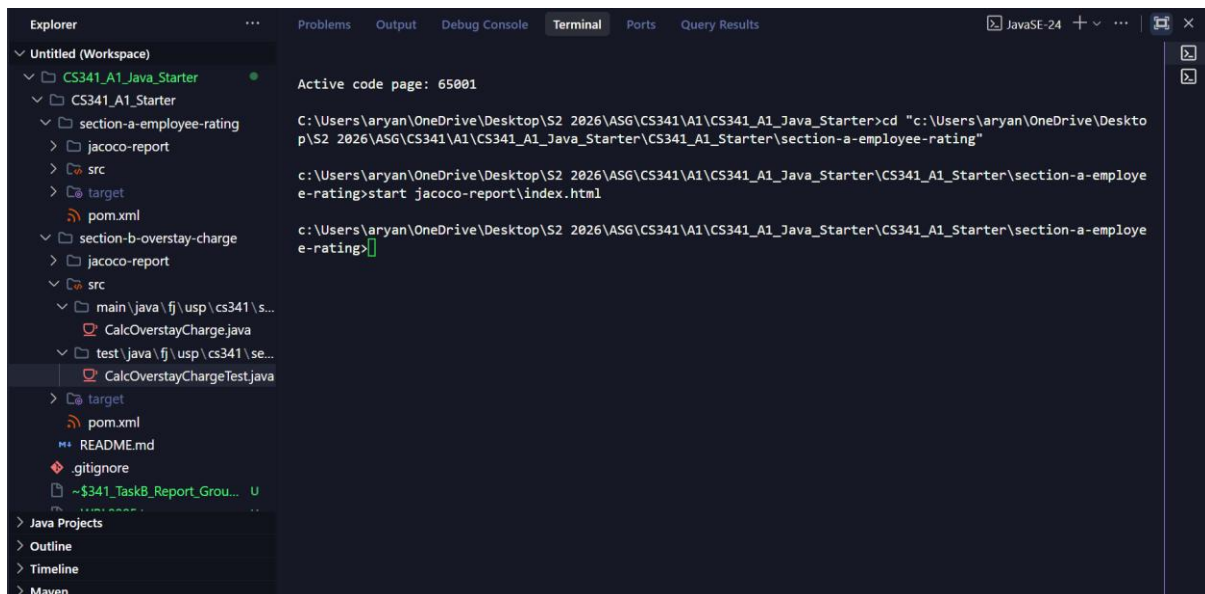
Bottom Screenshot: The terminal shows the execution of the command `mvn clean test` again. The output indicates that the project is being scanned for projects, and the build process is initiated. The output shows the following steps:

- Building CS341 Section B - Car Park Overstay Charge 1.0.0 from pom.xml
- Compiling 1 source file with javac [debug release 17] to target\classes
- Running tests using JUnit 4.12.0 and JUnit Platform 1.8.2
- Tests run: 43, Failures: 0, Errors: 0, Skipped: 0
- BUILD SUCCESS

JaCoCo Testing Logs

The test suite and JaCoCo coverage reports were generated by running the command: **mvn clean verify**. The generated report was then viewed in the browser using the command: **start target\site\jacoco\index.html** (or by opening jacoco-report\index.html).

The following screenshots are for the Section A code (section-a-employee-rating)



We were able to achieve a 100% branch & line coverage as shown in the pictures below.

CS341 Section A - Employee Performance Rating System [Sessions](#)

CS341 Section A - Employee Performance Rating System

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fj.usp.cs341.sectiona		100%		100%	0	15	0	20	0	6	0	1
Total	0 of 77	100%	0 of 18	100%	0	15	0	20	0	6	0	1

Created with JaCoCo 0.8.12.202403310830

CS341 Section A - Employee Performance Rating System > fj.usp.cs341.sectiona

[Source Files](#) [Sessions](#)

fj.usp.cs341.sectiona

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Employee		100%		100%	0	15	0	20	0	6	0	1
Total	0 of 77	100%	0 of 18	100%	0	15	0	20	0	6	0	1

Created with JaCoCo 0.8.12.202403310830

CS341 Section A - Employee Performance Rating System > fj.usp.cs341.sectiona > Employee

[Sessions](#)

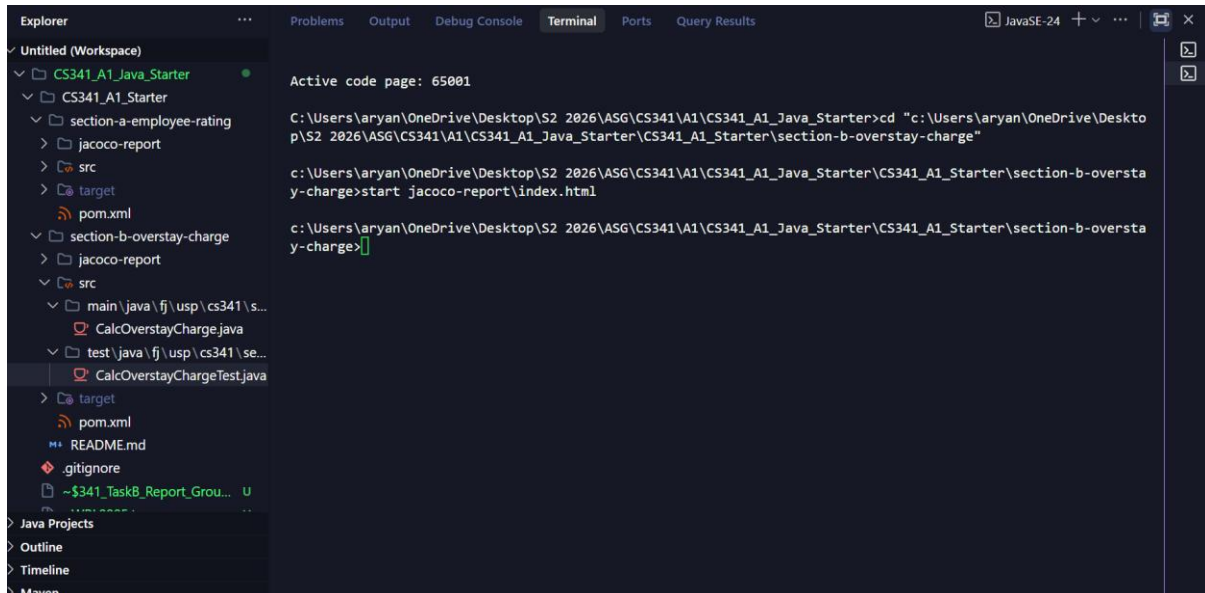
Employee

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Employee(String, int)		100%		100%	0	5	0	8	0	1
getRating()		100%		100%	0	4	0	7	0	1
getSummary()		100%		100%	0	2	0	2	0	1
isBonusEligible()		100%		100%	0	2	0	1	0	1
getName()		100%		n/a	0	1	0	1	0	1
getScore()		100%		n/a	0	1	0	1	0	1
Total	0 of 77	100%	0 of 18	100%	0	15	0	20	0	6

Created with JaCoCo 0.8.12.202403310830

The test suite and JaCoCo coverage reports were generated by running the command: **mvn clean verify**. The generated report was then viewed in the browser using the command: **start target\site\jacoco\index.html** (or by opening jacoco-report\index.html).

The following screenshots are for the Section B code (section-b-overstay-charge).



We were able to achieve a 100% branch & line coverage as shown in the evidence below.

CS341 Section B - Car Park Overstay Charge [Sessions](#)

CS341 Section B - Car Park Overstay Charge

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fj.usp.cs341.sectionb		100%		100%	0	18	0	21	0	3	0	1
Total	0 of 94	100%	0 of 30	100%	0	18	0	21	0	3	0	1

Created with JaCoCo 0.8.12.202403310830

CS341 Section B - Car Park Overstay Charge > [fj.usp.cs341.sectionb](#) [Source Files](#) [Sessions](#)

fj.usp.cs341.sectionb

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
CalcOverstayCharge		100%		100%	0	18	0	21	0	3	0	1
Total	0 of 94	100%	0 of 30	100%	0	18	0	21	0	3	0	1

Created with JaCoCo 0.8.12.202403310830

CS341 Section B - Car Park Overstay Charge > [fj.usp.cs341.sectionb](#) > [CalcOverstayCharge](#) [Sessions](#)

CalcOverstayCharge

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
computeOverstayCharge(int, int, int)		100%		100%	0	12	0	17	0	1		
isFlaggedForClamping(int, int, int)		100%		100%	0	5	0	3	0	1		
CalcOverstayCharge()		100%		n/a	0	1	0	1	0	1		
Total	0 of 94	100%	0 of 30	100%	0	18	0	21	0	3		

Created with JaCoCo 0.8.12.202403310830

Section B – Test Design, Execution and Defect Logging

Task 4 – Test case identification and Unit Test Plan

4.1 Test Design Approach

Equivalence Partitioning and Boundary Value Analysis were used to analyze the **computeOverstayCharger** functionality. Equivalence Partitioning was used to divide the input parameters into valid and invalid classes and to identify the different charging partitions. For **hoursOverstayed**, Boundary Value Analysis was applied around the overall valid range of 1 to 48 hours together with the charging transitions at 4/5 and 12/13 hours. The expected results were derived from the assignment specification before executing the implementations supplied.

4.2 Equivalence Partitioning

4.2.1 hoursOverstayed

For **hoursOverstayed** the valid range is 1 to 48 hours inclusive and values below 1 and above 48 are invalid.

Class ID	Equivalence Class	Validity
H1	hoursOverstayed < 1	Invalid
H2	$1 \leq \text{hoursOverstayed} \leq 48$	Valid
H3	hoursOverstayed > 48	Invalid

Since different charging rates are applied within the valid hours range, the valid range was divided into functional equivalence partitions.

Car With Visitor Permit

Class	Hours	Expected Behavior
HV1	1-4	\$ 2.00 per hour
HV2	5-12	\$4.00 per hour
HV3	13-48	\$8.00 per hour and flagged for clamping

Car with Resident Permit

Class ID	Hours	Expected Behavior
HR1	1-12	\$1.00 per hour
HR2	13-48	\$3.00 per hour

Truck

Class ID	Hours	Expected Behavior
HT1	1-48	\$15.00 per hour

4.2.2 vehicleType

Class ID	Value/Range	Description	Validity
V1	1	Car	Valid
V2	2	Truck	Valid
V3	< 1	Undefined Vehicle type	Invalid
V4	> 2	Undefined Vehicle type	Invalid

0 and 3 are the invalid values selected for testing

4.2.3 permitType

permitType Parameters uses integer codes. The valid value is 1 for Visitors and 2 for Residents, any other value is invalid.

Class ID	Value/Range	Description	Validity
P1	1	Visitor	valid
P2	2	Resident	valid
P3	< 1	Undefined Permit type	invalid
P4	> 2	Undefined permit type	invalid

The invalid values selected for testing are 0 and 3 and even though Trucks use the same hourly rate for either permit type, the specification defines only permit values 1 and 2 and state the method must return -1 if any input is invalid, that means invalid permits values must also be rejected for trucks.

4.3 Boundary Value Analysis

Boundary value analysis was applied to the **hoursOverstayed** input because the validity and charging rate change at specific hour boundaries. Values below, on and above important boundaries were selected.

Boundary	Test Values
Minimum Valid Value = 1	0, 1 and 2
Maximum Valid Value = 48	47, 48 and 49

Visitor Car 4/5 Hour Boundary

Position	Hours
Just below boundary	3
Upper value of first band	4
Lower value of second band	5
Just above boundary	6

Visitor car 12/13 Hour Boundary

Position	Hours
Just below boundary	11
Upper value of second band	12
Lower value of third band	13
Just above boundary	14

Resident Car 12/13 Hour Boundary

Position	Hours
Just below boundary	11
Upper value of first band	12
Lower value of second band	13
Just above boundary	14

For the vehicleType and permitType, Equivalence Partitioning will be the primary technique used because these inputs represent discrete categories rather than continuous numeric values. Values 0 and 3 were selected as invalid values which is immediately outside the valid values 1 and 2.

4.4 Unit Test Plan

All the identified Boundary Value Analysis and Equivalence Partitioning test cases are documented in the Unit Test Plan workbook. Test cases have unique test case ID, the partition being tested, test techniques, inputs, expected results, actual results and statuses. Expected results were derived before execution and the actual results and statuses were recorded during Task 5.

Conclusion

A complete test suite of 35 test cases (comprising Boundary Value Analysis and Equivalence Partitioning scenarios) was executed against the initially supplied CalcOverstayCharge implementation. During the baseline execution in Task 5, 29 test cases passed and 6 test cases failed, exposing 4 distinct root-cause defects logged as DF-01 through DF-06 (affecting test cases TC-11, TC-16, TC-20, TC-29, TC-34, and TC-35).

In Task 6, all 6 defects were remediated in source code by correcting the tier boundary conditions, implementing the missing upper limit constraint (hoursOverstayed > 48), and placing permit validation prior to vehicle branching. Upon regression testing with the automated JUnit 5 test suite, all 35 test cases achieved 100% PASSED status, 100% line and branch coverage was verified via JaCoCo, and all defect entries in the Defect Tracker were marked as Closed.